MSDM5004
Numerical Methods and Modeling in Science
Spring 2026

Lecture 14

Prof Yang Xiang
Hong Kong University of Science and Technology

Chapter 11

Introduction to neural network methods for solving PDEs

Neural network approximation of a function

A neural network (fully connected)

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{W}_N \boldsymbol{\sigma}(\mathbf{W}_{N-1}\boldsymbol{\sigma}(\cdots \boldsymbol{\sigma}(W_2\boldsymbol{\sigma}(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2)\cdots+\mathbf{b}_{N-1})+\mathbf{b}_N$$

$\boldsymbol{\theta}$ denotes all the parameters $\sigma(\cdot)$ is the activation function

$\mathbf{x}$: input (vectors)
$\mathbf{y}$: output (vectors)
N: number of hidden layers

w_k: weights (matrix)
b_k: bias (vector)

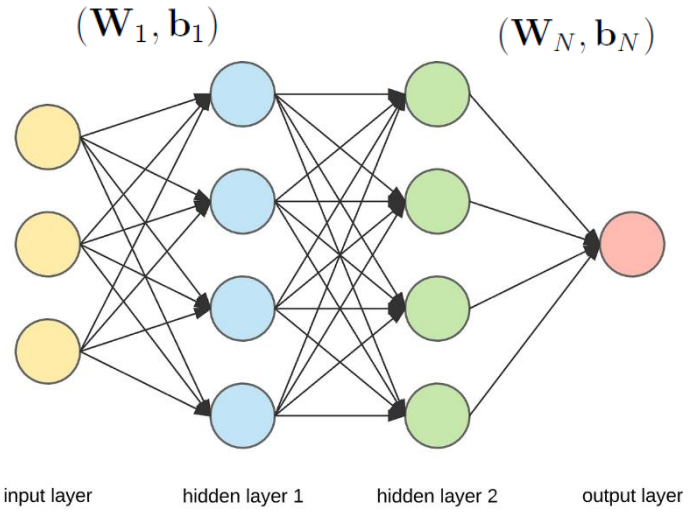
$$\boldsymbol{\sigma}(\mathbf{x}) = \begin{pmatrix} \sigma(x_1) \\ \sigma(x_2) \\ \cdots \\ \sigma(x_d) \end{pmatrix}$$

Two layer neural network

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = W_2\boldsymbol{\sigma}(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2$$

Universal approximation theorem [1]

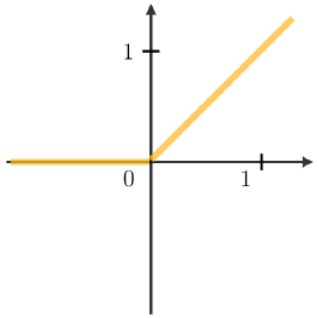
$$f_{\boldsymbol{\theta}}(\mathbf{x}) \approx f(\mathbf{x})$$



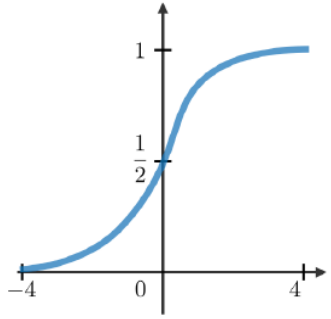
[1] K. Hornik, M. Stinchcombe, and H. White, Multilayer feedforward networks are universal approximators, Neural networks, 2 (1989), pp. 359–366.

Commonly used activation function

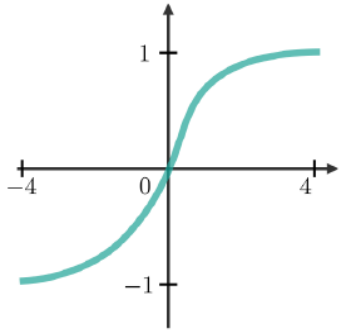
ReLU (Retified Linear Unit): $\sigma(x) = \max\{x, 0\}$



Sigmoid: $\sigma(x) = \dfrac{1}{1+e^{-x}}$



Tanh: $\sigma(x) = \dfrac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$



Supervised learning: trained from labeled data

Determining the neural network parameters $\boldsymbol{\theta}$ by the optimization problem:

Minimize the loss function $\mathcal{L}(\boldsymbol{\theta})$

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{M}\sum_{m=1}^{M} l\left(y_{\boldsymbol{\theta}}^{m}, y_{g}^{m}\right)$$

$\mathbf{y}_g$: ground truth

Example of the loss function

Mean squared error (MSE) $\quad \frac{1}{M}\sum_{m=1}^{M}\left(y_{\boldsymbol{\theta}}^{m} - y_{g}^{m}\right)^{2}$

Training

Solving the optimization problem by gradient descent (GD)

$$\theta_k^{n+1} = \theta_k^n - \frac{\alpha}{M} \sum_{m=1}^{M} \left. \frac{\partial l\left(y_{\boldsymbol{\theta}}^m, y_g^m\right)}{\partial \theta_k} \right|_{\boldsymbol{\theta}=\boldsymbol{\theta}^n} \qquad \text{for each } k$$

α: learning rate

Computed by backpropagation (*k* from *N* to *0*) by chain rule formulation

- epoch: one full, complete pass of the entire training dataset through the neural network

Compute partial derivatives by the chain rule

$$\mathbf{y} = w_N \boldsymbol{\sigma}(w_{N-1}\boldsymbol{\sigma}(\cdots(w_2\boldsymbol{\sigma}(w_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)\cdots + \mathbf{b}_{N-1}) + \mathbf{b}_N)$$

Automatic differentiation

Neural network approximation of *f(x)*

$$f_{\boldsymbol{\theta}}(\mathbf{x}) \approx f(\mathbf{x})$$

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{W}_N \boldsymbol{\sigma}(\mathbf{W}_{N-1}\boldsymbol{\sigma}(\cdots \boldsymbol{\sigma}(W_2\boldsymbol{\sigma}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots + \mathbf{b}_{N-1}) + \mathbf{b}_N$$

Automatic differentiation

$$\frac{\partial f}{\partial x_i} \approx \frac{\partial f_{\boldsymbol{\theta}}}{\partial x_i}$$

Also computed by backpropagation (*k* from *N* to *0*) by chain rule formulation

Stochastic gradient decent (SGD)

- For each iteration, a small random subset of data (a mini-batch) is selected.
- The gradient of the loss function is calculated with respect to the model's parameters for only that mini-batch.

[2] L. Bottou, Large-Scale Machine Learning with Stochastic Gradient Descent, Proceedings of COMPSTAT 2010, Paris, 22-27 August 2010, 177-186.

Adaptive moment estimation (Adam)

- Computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients.

[3] D. P. Kingma, J. Ba, Adam: A Method for Stochastic Optimization, ICLR 2015.

Physically informed neural networks (PINN)

A PDE, e.g.

$$u_t + \mathcal{N}[u] = f(\mathbf{x}, t), \ \mathbf{x} \in \Omega, \ t \in [0, T]$$

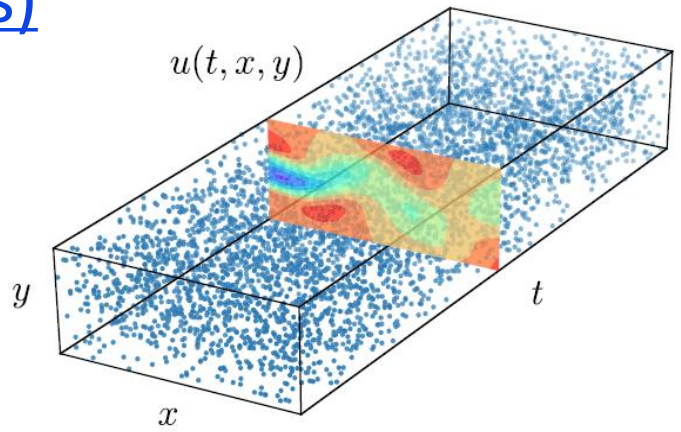
where $\mathcal{N}[\cdot]$ is a differential operator in space.

For diffusion equation in two dimensions, $\mathcal{N}[u] = -u_{xx} - u_{yy}, \ f(\mathbf{x}, t) = 0.$

Sampling points (collocation points)

$$(\mathbf{x}_i, t_i), \ i = 1, 2, \cdots, N$$

can be random



[4] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, Journal of Computational Physics, 378, 686–707, 2019.

[5] GE Karniadakis, IG Kevrekidis, L Lu, P Perdikaris, S Wang, L Yang, Physics-informed machine learning, Nature Reviews Physics 3 (6), 422-440, 2021.

Loss

Initial-boundary value problem of the PDE

$$u_t + \mathcal{N}[u] = f(\mathbf{x}, t), \ \mathbf{x} \in \Omega, \ t \in [0, T]$$

$$u = g, \ \mathbf{x} \in \partial\Omega$$

$$u = u_0, \ t = 0$$

$\{(\mathbf{x}_f^i, t_f^i)\}$: points inside the domain
$\{(\mathbf{x}_b^i, t_b^i)\}$: points on $\partial\Omega$
$\{(\mathbf{x}_0^i, 0)\}$: points at $t = 0$

Mean squared error

$$\mathrm{MSE} = \mathrm{MSE}_f + \mathrm{MSE}_0 + \mathrm{MSE}_b$$

$$\mathrm{MSE}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| u_t(\mathbf{x}_f^i, t_f^i) + \mathcal{N}[u](\mathbf{x}_f^i, t_f^i) - f(\mathbf{x}_f^i, t_f^i) \right|^2$$

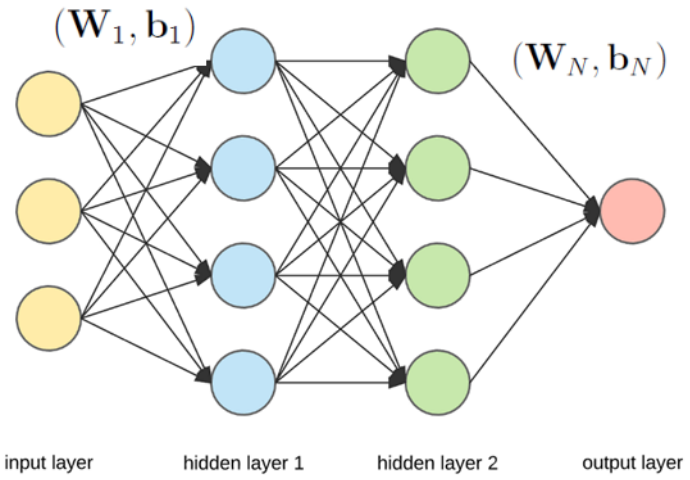
$$\mathrm{MSE}_b = \frac{1}{N_b} \sum_{i=1}^{N_b} \left| u(\mathbf{x}_b^i, t_b^i) - g(\mathbf{x}_b^i, t_b^i) \right|^2$$

$$\mathrm{MSE}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} \left| u(\mathbf{x}_0^i, 0) - u_0(\mathbf{x}_0^i) \right|^2$$

unsupervised learning
(not based on data)

Neural network structure

Fully-connected neural network



2-5 layers

Example

Schrödinger equation

$$ih_t + 0.5h_{xx} + |h|^2 h = 0, \quad x \in [-5, 5], \quad t \in [0, \pi/2],$$

$$h(0, x) = 2\,\mathrm{sech}(x),$$

$$h(t, -5) = h(t, 5),$$

$$h_x(t, -5) = h_x(t, 5),$$

$N_f = 20000$ points, $N_b = 50$ points, $N_0 = 50$ points, random

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| ih_t(t_f^i, x_f^i) + 0.5h_{xx}(t_f^i, x_f^i) + |h(t_f^i, x_f^i)|^2 h(t_f^i, x_f^i) \right|^2$$

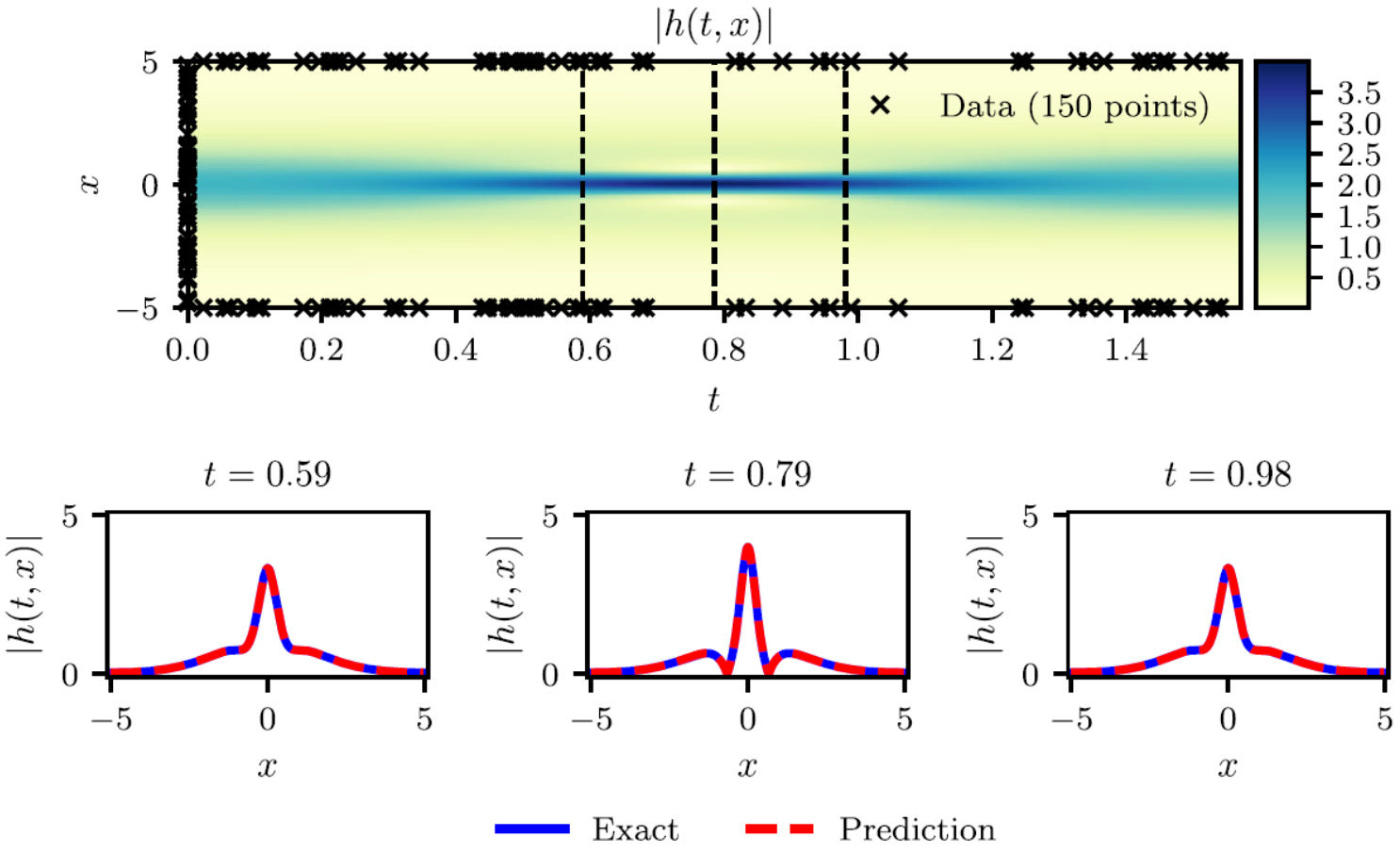
$$MSE_b = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(|h^i(t_b^i, -5) - h^i(t_b^i, 5)|^2 + |h_x^i(t_b^i, -5) - h_x^i(t_b^i, 5)|^2 \right)$$

$$MSE_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |h(0, x_0^i) - h_0^i|^2$$

$$h(t, x) = \begin{bmatrix} u(t, x) & v(t, x) \end{bmatrix}$$

- 5-layer deep neural network with 100 neurons per layer
- a hyperbolic tangent activation function

Results



Inverse problem

Navier–Stokes equation

$$u_t + \lambda_1(uu_x + vu_y) = -p_x + \lambda_2(u_{xx} + u_{yy}),$$
$$v_t + \lambda_1(uv_x + vv_y) = -p_y + \lambda_2(v_{xx} + v_{yy}),$$

where $u(t, x, y)$ denotes the x-component of the velocity field, $v(t, x, y)$ the y-component, and $p(t, x, y)$ the pressure.

$\lambda = (\lambda_1, \lambda_2)$ are the unknown parameters

Assume $u = \psi_y, \quad v = -\psi_x$

to satisfy the incompressible condition $u_x + v_y = 0$

approximating $[\psi(t, x, y) \quad p(t, x, y)]$ using a single neural network with two outputs.

Inverse problem:

Given noisy measurements $\{t^i, x^i, y^i, u^i, v^i\}_{i=1}^N$ to determine $\lambda = (\lambda_1, \lambda_2)$

Denote

$$f := u_t + \lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy}),$$
$$g := v_t + \lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy}),$$

The mean squared error is

$$MSE := \frac{1}{N}\sum_{i=1}^{N}\left(|u(t^i, x^i, y^i) - u^i|^2 + |v(t^i, x^i, y^i) - v^i|^2\right) \cdot$$

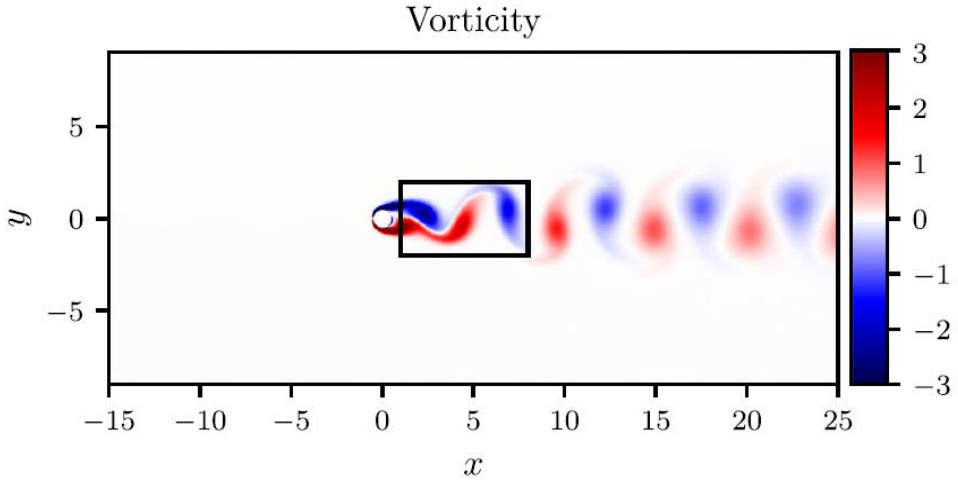
$$+ \frac{1}{N}\sum_{i=1}^{N}\left(|f(t^i, x^i, y^i)|^2 + |g(t^i, x^i, y^i)|^2\right)$$

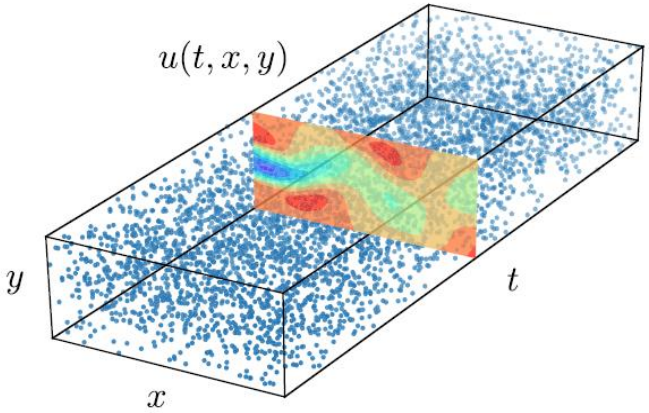
Train the neural network to find $\begin{bmatrix}\psi(t,x,y) & p(t,x,y)\end{bmatrix}$

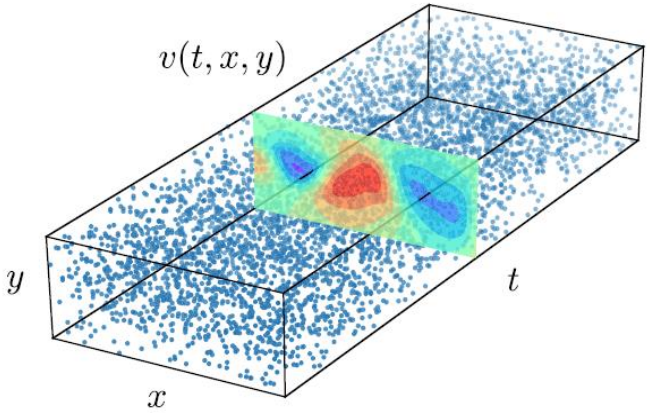
and $\lambda = (\lambda_1, \lambda_2)$

9 layers with 20 neurons in each layer

Results:







$N = 5{,}000$

Correct PDE	$u_t + (uu_x + vu_y) = -p_x + 0.01(u_{xx} + u_{yy})$ $v_t + (uv_x + vv_y) = -p_y + 0.01(v_{xx} + v_{yy})$
Identified PDE (clean data)	$u_t + 0.999(uu_x + vu_y) = -p_x + 0.01047(u_{xx} + u_{yy})$ $v_t + 0.999(uv_x + vv_y) = -p_y + 0.01047(v_{xx} + v_{yy})$
Identified PDE (1% noise)	$u_t + 0.998(uu_x + vu_y) = -p_x + 0.01057(u_{xx} + u_{yy})$ $v_t + 0.998(uv_x + vv_y) = -p_y + 0.01057(v_{xx} + v_{yy})$

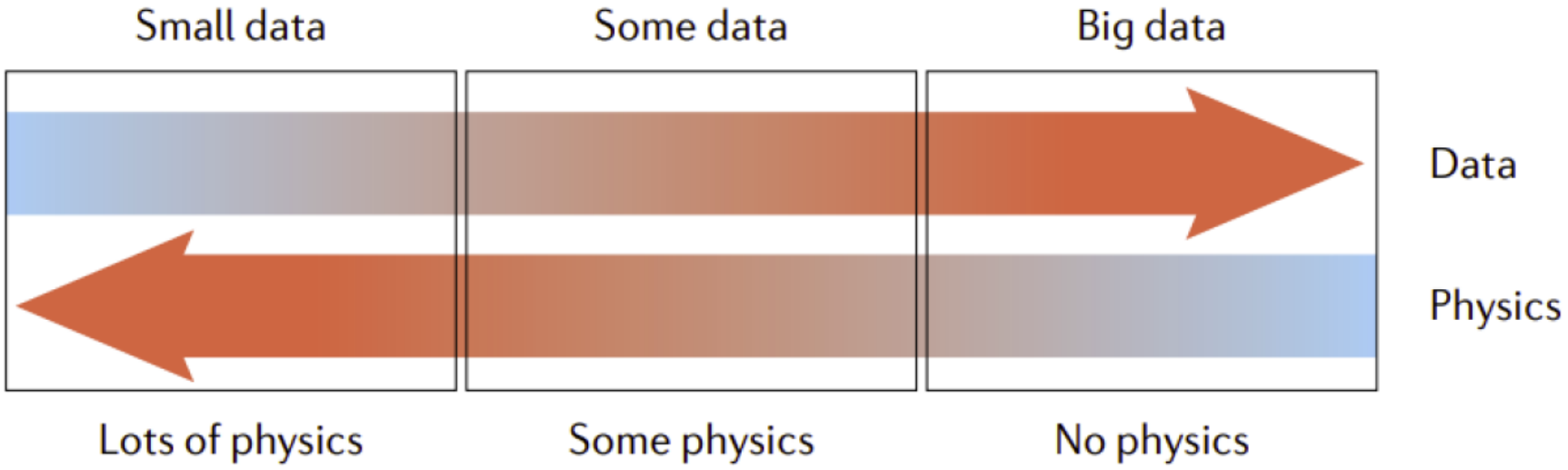
Neural network/numerical solutions of PDEs

- In low dimensions ($d \leq 3$)
  - Traditional numerical methods typically work well
  - Deep learning methods: high coding efficiency

    suitable for inverse problems

- In high dimensions
  - Tradition methods fail (Curse of dimensionality)
  - Deep learning methods work

Three possible categories of physical problems and associated available data



- Small data regime: we know all the physics and coefficients of PDE, as well as data for initial and boundary conditions.

- The middle one: we know some data and some physics, possibly miss some parameters or even an entire term in PDE.

- Big data regime: we may not know any of the physics, but we have a big amount of data. Then a data-driven approach may be most effective. For example, using operator regression methods to discover new physics.